

Things Smaller than Words

David R. Mortensen

January 13, 2026

Introduction

When we want to process language, we need to represent it as numbers and, since we assume that it is linear, we usually want to represent it as a sequence of numbers. But how should a passage of text be represented? In particular, how should it be divided into units that can be processed computationally?

To many people, the most obvious unit into which text can be divided is the word. In this scheme, the utterance

(1) Words aren't simply atomic entities!

would have five basic units. As we will learn, defining `WORD` is not trivial. Even in our example, there is a complication: *aren't* can be seen as either a single word or as the combination of two words, *are* + *not*. In this instance, *n't* is a special kind of word called a clitic, which latches on to a neighboring word. If we segment off *n't*, then we get a sequence of six units:

(2) Words are n't simply atomic entities!

This has the benefit of giving all of the texts that have *n't* similar representations, even when *n't* is part of *didn't*, *isn't*, *wasn't*, *weren't*, and so on. It effectively reduces the sparsity of the text data, which is good, but it does not scratch the surface of the problem. The frequencies of words roughly follow a power law distribution. A few words are very frequent, but most words are very infrequent. This means that we cannot learn a good representation of most words if we are building models using words as tokens. It would be nice, both from a computational and a theoretical perspective, if we could analyze text into units that did not display this kind of distribution.

The obvious answer is to tokenize by letter or character.

(3) W o r d s _ a r e n ' t _ s i m p l y _ a t o m i c _ e
n t i t i e s !

While this may be an obvious choice for languages written with `LOGO-GRAPHIC` writing systems like Chinese (where the number of possible characters is very large and where the distribution of characters is roughly Zipfian), it addresses some aspects of tokenization for languages written with `ALPHABETIC` systems even better. English, for example, has only 26 letters plus a handful of punctuation and whitespace characters. Every word (with exceptions like emoji) will be made up of strings of these characters.

One might even tokenize by phonemes (individual units of sound), in which case the sequence would look like:

(4) w ɹ d z a ɹ n t s i m p l i a t a m i k e n t i t i z

This is known as Zipf's Law, stated simply as $f(r) \cong \frac{0.1}{r}$, where r is the rank order of a word. In other words, the frequency of the word with rank order r is approximately $0.1/r$.

Duality of Patterning

But character-level and phoneme-level tokenizations run up against a problem. As Charles Hockett noted in his classic textbook, *A Course in Modern Linguistics*, language and other sign systems display “duality of patterning”¹. The observation is that the smallest units in language that can contribute to making distinctions in meaning are (by themselves) meaningless. For alphabetic writing, these are letters; for speech, these are phonemes. A similar observation was made by Martinet². The upshot, from a computational perspective, is that embeddings of $\langle t \rangle$ ³ are never going to be as meaningful as embeddings of tokens like $\langle n't \rangle$. $\langle n't \rangle$ only occurs in sentences that include NEGATIVE as one of their components, but $\langle t \rangle$ can occur in almost any sentence.

Duality of patterning states that there is a basic level of units that are distinctive but meaningless and that there is a more complex level of units that are both meaningful and possess internal structure (that is, they are made out of units from the basic level). If we could tokenize text into units on the second level, each token would be meaningful and could be assigned a meaningful embedding (e.g., in the encoder of a transformer). In principle, in fact, we should be able to tokenize a sentence so that each token is maximally informative with respect to the meaning of the sentence.

Tokenization Schemes

In NLP, various means have been developed to tokenize text below the level of the word, all of which will be discussed in greater detail subsequently. The simplest of these is byte pair encoding, a version of Huffman coding introduced to NLP by Rico Sennrich⁴. Like the subsequent tokenization algorithms mentioned here, it is sensitive to the hyperparameter VOCABULARY SIZE. Our sentence, when tokenized with the BPE model employed with the GPT-4 model, is as follows:

(5) Words _aren 't _simply _atomic _entities !

Here, _ stands for a space. WordPiece is a closely related tokenization scheme⁵. The WordPiece model used in BERT yields similar results:

(6) words aren ' t simply atomic entities !

SentencePiece is a rather different tokenization scheme (actually, two schemes) that does not assume that words are separated by whitespace⁶. The same sentence tokenized by the SentencePiece tokenizer used with certain versions of RoBERTa looks like this:

(7) _ Words _ aren't _ simply _ atomic _ entities!

That is, except for the treatment of whitespace, it looks just like Example (1). SentencePiece is based in part on a Unigram model of tokenization. When

¹ Charles F. Hockett. *A course in modern linguistics*. Macmillan, New York, 1958

² André Martinet. *Éléments de linguistique générale*. Colin, Paris, 1960

³ Letters enclosed in angle brackets are GRAPHEMES—units of written language.

Meaningful units and how they combine will be discussed more in Lecture 2.

Tokenization algorithms will be discussed in detail in Lecture 6.

⁴ Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith, editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. DOI: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162>

⁵ Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016

⁶ Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics. DOI: 10.18653/v1/D18-2012. URL <https://aclanthology.org/D18-2012>

applied to our sentence, Unigram yields results like:

(8) _Word s _are n ' t _simply _atomic _entitie s !

The dramatic difference seen here between SentencePiece and Unigram is due in part to differences in the vocabulary size of the models rather than to the algorithms themselves. Nevertheless, it provides a convenient opportunity for comparison.

Morphemes

In the Unigram output, you probably noticed something you did not notice in the output of the earlier tokenizers: the ⟨s⟩ at the ends of ⟨Words⟩ and ⟨entities⟩ is segmented off as a separate token. Also, ⟨are⟩ is treated as a token independent of ⟨n⟩, ⟨'⟩, and ⟨t⟩ (which follow it). This is because Unigram captures an insight from linguistics that is not captured by the BPE, Word-Piece, or SentencePiece BPE tokenizers: ⟨Word⟩, ⟨are⟩, and ⟨entitie⟩ are all MORPHEMES (the units on Hockett's second tier). The plural ⟨s⟩ suffixes are also morphemes.

When tokenized into morphemes, our sentence would be as in (9):

(9) _Word s _are n't _simply _atomic _entitie s

Some observations:

- (10) *A morpheme is a minimal meaningful unit.*
 - a. By definition, a morpheme is something that can have a meaningful embedding.
 - b. All words are composed of morphemes.
 - c. Rare words are usually composed of more frequent morphemes.
- (11) Working hypothesis:
 - a. An optimal tokenization is one in which each token consists of exactly 1 morpheme.
 - b. The next most optimal tokenizations are those in which each token consists of exactly n morphemes.

Allomorphy

Note something interesting about *simply* and *entities* in (9) above:

- (12) a. The root *simple* is spelled here as ⟨simp⟩ (followed by the suffix ⟨ly⟩).
- b. The root *entity* is spelled here as ⟨entitie⟩ (followed by the suffix ⟨s⟩).

These are examples of the phenomenon called ALLOMORPHY⁷.

Hypothesis:

- (13) In an optimal tokenization, tokens are normalized such that all instances of a particular morpheme have the same representation (and the same embedding).

⁷ Allomorphy is when the same morpheme has different spellings or pronunciations based on context. It will be discussed more in Lecture 7.

Looking Forward

In this lecture, we have already briefly introduced several of the themes of this module:

- Meaningful units (signs) below the level of the word (morphemes)
- Morphological segmentation
- Computational approaches to segmentation and their relationship to tokenization
- Allomorphy

We will also talk about a number of related topics:

- Inflection, derivation, and compounding (morphological functions)
- Morphological operations on form besides affixation

AFFIXATION is a general term for prefixation and suffixation.

The First Project

The goal of the first project is to develop a tokenizer—a morphological segmenter—that segments text from an arbitrary language more similarly to gold morpheme segmentations produced by a linguist than three baselines: a rule-based (FST) baseline, a Unigram tokenization model, and a BPE tokenizer with vocabulary size tuned on the development set.

The data

You will be provided with data from two languages:

- Rarámuri (Tarahumara) [tar], an Indigenous language of northern Mexico that is part of the Uto-Aztecan language family.
- Shipibo-Konibo [shp], a Panoan language of the Peruvian Amazon.

For each language and for each split (train, dev, and test), you will be provided with a source file. You will be provided with the target file for the train and dev sets⁸. The source file will consist of unsegmented words, one per line. The target file will consist of the same words in the same order, but with the morphemes delimited by spaces, as shown in (14).

⁸ The test target file will be held out.

(14) The first entries from the Shipibo-Konibo source and target data

- a. src: ointiyamaai
- b. tgt: ointi yama ai

The task

The goal is to reproduce the reference target given the source and a model trained on the (source side of the) training set. This can be seen as a sequence labeling task in which your model must decide whether a character is the beginning of a new span (morpheme) or the continuation of an existing span. There are other ways of viewing the task, but evaluation (in terms of precision, recall, and F1) will be based on this paradigm. Labeling a character as “beginning” will be treated as a positive, and not labeling it as such will be treated as a negative.

Baselines

As will often be the case in this course, there will be two baselines: one data-driven and one rule-based.

Morfessor (data-driven). For each language, the Morfessor morphological segmentation algorithm will be applied. This algorithm, which is reasonably complicated, is designed to yield segmentations that resemble morpheme segmentations used by linguists.

Unigram (data-driven). For each language, a Unigram model will be trained on the training set. If other unlabeled text data can be found, it will be used as well (and shared with you). You will be provided with the Unigram tokenizer output for the dev set.

BPE (data-driven). For each language, a BPE model will be trained on the training set. If other unlabeled text data can be found, it will be used as well (and shared with you). Vocabulary size will be tuned on the development set. You will be provided with the BPE tokenizer output for the dev set.

Suggestions

- Develop improved information-theoretic tokenization algorithms in the same family as BPE or Unigram tokenization.
- Develop better means of selecting an optimal vocabulary for BPE-like algorithms⁹.
- Improve on unsupervised morphological segmentation algorithms like Morfessor and Morfessor FlatCat¹⁰.
- Build a better rule-based (probably FST-based) morphological segmenter, perhaps using LEXC and Foma¹¹.

Extensions

- Does your segmentation or tokenization method provide better performance on some downstream tasks than widely used tokenization schemes like BPE, Unigram, and SentencePiece?

⁹ Jingjing Xu, Hao Zhou, Chun Gan, Zaixiang Zheng, and Lei Li. Vocabulary learning via optimal transport for neural machine translation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7361–7373, Online, August 2021. Association for Computational Linguistics. DOI: 10.18653/v1/2021.acl-long.571. URL <https://aclanthology.org/2021.acl-long.571>

¹⁰ Stig-Arne Grönroos, Sami Virpioja, Peter Smit, and Mikko Kurimo. Morfessor FlatCat: An HMM-based method for unsupervised and semi-supervised learning of morphology. In Junichi Tsujii and Jan Hajic, editors, *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 1177–1185, Dublin, Ireland, August 2014. Dublin City University and Association for Computational Linguistics. URL <https://aclanthology.org/C14-1111>; and Stig-Arne Grönroos, Sami Virpioja, and Mikko Kurimo. Morfessor EM+Prune: Improved subword segmentation with expectation maximization and pruning. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 3944–3953, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL <https://aclanthology.org/2020.lrec-1.486>

¹¹ Mans Hulden. Foma: a finite-state

- More generally, is there a correlation between how well a tokenization scheme performs on the morphological segmentation task and how well it performs on downstream NLP tasks?
- How does the runtime efficiency of your method compare with the other methods?

References

- Stig-Arne Grönroos, Sami Virpioja, Peter Smit, and Mikko Kurimo. Morfessor FlatCat: An HMM-based method for unsupervised and semi-supervised learning of morphology. In Junichi Tsujii and Jan Hajic, editors, *Proceedings of COLING 2014, the 25th International Conference on Computational Linguistics: Technical Papers*, pages 1177–1185, Dublin, Ireland, August 2014. Dublin City University and Association for Computational Linguistics. URL <https://aclanthology.org/C14-1111>.
- Stig-Arne Grönroos, Sami Virpioja, and Mikko Kurimo. Morfessor EM+Prune: Improved subword segmentation with expectation maximization and pruning. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 3944–3953, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL <https://aclanthology.org/2020.lrec-1.486>.
- Charles F. Hockett. *A course in modern linguistics*. Macmillan, New York, 1958.
- Mans Hulden. Foma: a finite-state compiler and library. In Jörn Kreutel, editor, *Proceedings of the Demonstrations Session at EACL 2009*, pages 29–32, Athens, Greece, April 2009. Association for Computational Linguistics. URL <https://aclanthology.org/E09-2008>.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In Eduardo Blanco and Wei Lu, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, Brussels, Belgium, November 2018. Association for Computational Linguistics. DOI: 10.18653/v1/D18-2012. URL <https://aclanthology.org/D18-2012>.
- André Martinet. *Éléments de linguistique générale*. Colin, Paris, 1960.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In Katrin Erk and Noah A. Smith,

editors, *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, August 2016. Association for Computational Linguistics. DOI: 10.18653/v1/P16-1162. URL <https://aclanthology.org/P16-1162>.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Jingjing Xu, Hao Zhou, Chun Gan, Zaixiang Zheng, and Lei Li. Vocabulary learning via optimal transport for neural machine translation. In Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli, editors, *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 7361–7373, Online, August 2021. Association for Computational Linguistics. DOI: 10.18653/v1/2021.acl-long.571. URL <https://aclanthology.org/2021.acl-long.571>.